Breaking GUI Agent Actions: Realistic Adversarial Attacks on Multimodal LLMs

KANGHUA MO, Guangzhou University, China
ZHENGDAO LI*, School of Artificial Intelligence, Guangzhou University, China
SHANSHAN HUANG†, School of Artificial Intelligence, Guangzhou University, China
LI HU, Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, China

Recent progress in Multimodal Large Language Models (MLLMs) has enabled GUI agents that interpret interface screenshots and execute natural-language instructions. However, their increasing role in human–computer interaction raises security concerns. Existing work shows that MLLMs are highly sensitive to interface perturbations, yet most attacks assume unrealistic capabilities such as direct access to model-ready inputs. In real deployments, attackers cannot control the screenshot pipeline or the non-differentiable preprocessing operations preceding inference, rendering many prior attacks ineffective. We introduce **RAA**, a realistic adversarial attack framework that models the complete screenshot-to-prediction pipeline. To address gradient blockage from non-differentiable preprocessing (e.g., resizing, quantization), we develop a *dual-branch differentiable preprocessing module* that restores gradient flow while remaining faithful to the actual inference path. To improve robustness against query phrasing changes, we further incorporate a *task-level semantic variation mechanism* that jointly optimizes over paraphrased task variants. Experiments on Qwen2.5-VL-3B/7B-Instruct and GUI-Owl-7B show that RAA consistently outperforms prior attacks, improving attack success rates by an average of 30% under localized perturbations and a 10.7% under global perturbations, while maintaining strong effectiveness against standard defenses such as resizing, compression, and noise. Ablation studies highlight the roles of perturbation region and semantic diversity. Our framework establishes a practical threat model for GUI agents and offers a foundation for future robustness and security research.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; • **Security and privacy** → **Domain-specific security and privacy architectures**.

Additional Key Words and Phrases: Multimodal large language models, GUI agents, Adversarial attacks, Differentiable preprocessing, Semantic variation, Model robustness, Interface security

1 Introduction

Multimodal Large Language Models (MLLMs) have recently demonstrated remarkable capabilities in understanding web pages and application interfaces [3, 41]. They can infer structural information from interface screenshots, identify interactive elements, and map natural-language instructions to precise click locations [25],

*Corresponding author
†Corresponding author

Authors' Contact Information: Kanghua Mo, mokanghua@gmail.com, Guangzhou University, Guangzhou, China; Zhengdao Li, cuterzhengdao@gmail.com, School of Artificial Intelligence, Guangzhou University, Guangzhou, China; Shanshan Huang, florahuangss@gzhu.edu.cn, School of Artificial Intelligence, Guangzhou University, Guangzhou, China; Li Hu, lily23.hu@polyu.edu.hk, Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong, China.

ACM 1551-6865/2026/6-ART
https://doi.org/10.1145/3821640

enabling a wide range of GUI agent applications such as web automation, intelligent assistants, and accessibility enhancement [6, 10, 21].

However, as MLLMs become increasingly embedded in human–computer interaction pipelines, their security risks have become more prominent. Prior work shows that these models are susceptible to a variety of interface-level interferences during visual perception and reasoning. On one hand, explicit malicious pop-ups or injected instructions can divert the model's attention and induce incorrect actions [16, 20, 37]; on the other hand, subtle and implicit visual perturbations may also shift click locations and lead to task failures [2, 26, 28, 30, 38]. Such attacks not only undermine interaction accuracy and robustness but may also trigger high-risk operations—e.g., clicking an unintended payment button or bypassing security checkpoints—introducing severe security concerns.

Despite growing evidence of MLLMs' vulnerability in interface understanding, systematic adversarial analyses of GUI agent systems remain scarce. Under realistic deployment conditions, attackers can be broadly categorized into two types. A *global attacker* (e.g., developers or privileged maintainers) has full control over interface layout and content, enabling perturbations at arbitrary regions of the screen [20, 28]. In contrast, a *local attacker* (e.g., ordinary users or merchants) can only modify specific interface components such as advertisements, product images, or shareable thumbnails, injecting seemingly benign visual content that becomes part of the locally captured screenshot [26, 30]. Our study considers both threat models and evaluates GUI agent robustness under varying attacker capabilities. Importantly, although local attackers have more limited privileges, their threats are more realistic and covert: a single well-crafted image embedded in the user-side interface can reliably mislead the agent without altering the global layout. Moreover, both global and local perturbations evade common anomaly-detection and permission-control mechanisms, creating high-risk attack surfaces closely aligned with real-world application scenarios.

A key assumption widely overlooked in prior work is that most existing methods construct adversarial perturbations directly on the *preprocessed, model-ready input*, implicitly assuming that the attacker can manipulate the final screenshot consumed by the model [38]. In real-world interactions, however, screenshots are captured on the client side before being sent to the agent, meaning the attacker cannot control the screenshot pipeline or interfere with the subsequent preprocessing steps [2, 28]. Consequently, any realistic attack must traverse the entire pipeline from "user-side screenshot → data preprocessing → model inference." Attacks optimized solely on the post-preprocessing format—although effective in controlled settings—often fail after resizing, quantization, and other preprocessing operations, rendering them ineffective in realistic GUI-agent deployments (see Figure 1). While a few recent efforts have begun to acknowledge this pipeline discrepancy [2, 28], a systematic and reliable solution is still lacking.

To address these challenges, we propose a *realistic adversarial attack* (RAA) for GUI agents. We explicitly model the full pipeline from the original interface to model inference and perform white-box end-to-end optimization over it. The key obstacle is that numerous preprocessing operators—such as nonlinear resizing and discrete quantization—are non-differentiable, causing gradient blockage and preventing direct optimization of perturbation placement and shape. To overcome this, we design a *dual-branch differentiable preprocessing mechanism*: it strictly reproduces the official inference path while introducing a differentiable surrogate branch aligned with the real path to restore gradient flow, with a verification branch ensuring perturbation validity under the real inference process. Furthermore, given the semantic diversity and dynamism of GUI agent environments, we develop a *task semantic variation mechanism* that incorporates semantically similar but differently phrased tasks during optimization, mitigating instruction-specific overfitting and improving attack robustness across task variations.

We conduct extensive evaluations on three representative models, Qwen2.5-VL-3B/7B-Instruct [4] and GUI-Owl-7B [33]. Experimental results show that our method significantly outperforms existing baselines across multiple settings, achieving on average a 30% improvement under localized perturbations and a 10.7% improvement under global perturbations compared with the strongest baseline. Additional robustness tests demonstrate that the

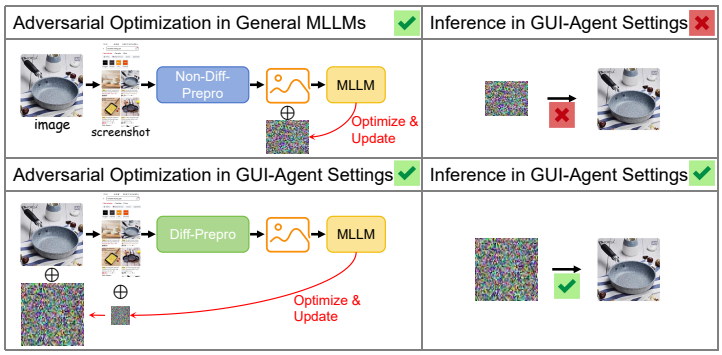

Fig. 1. Illustration of the full screenshot–differentiable preprocessing–inference pipeline and the failure of attacks optimized only on post-preprocessed inputs.

attack remains effective under various defense strategies, while ablation studies systematically analyze the effects of task semantic variation and perturbation region.

In summary, our contributions are threefold:

- We propose a realistic GUI agent adversarial attack formulation that unifies global and local attacker models and reveals a common but unrealistic assumption in existing work regarding screenshot and preprocessing control.
- We design a *dual-branch differentiable preprocessing mechanism* and a *task semantic variation mechanism* to address gradient masking and semantic overfitting, ensuring perturbations remain optimizable along the real inference pipeline and robust across semantic variations.
- We conduct systematic evaluations on mainstream MLLMs (e.g., Qwen2.5-VL-3B/7B-Instruct and GUI-Owl-7B), showing that our method significantly improves attack success rates and robustness over prior baselines, and we provide ablation studies highlighting the roles of perturbation region and semantic diversity.

2 Related Work

2.1 Multimodal Large Language Models and Security

Multimodal Large Language Models (MLLMs) have achieved substantial progress in cross-modal understanding, reasoning, and generation tasks in recent years. Despite these achievements, a growing body of research has shown that MLLMs remain highly vulnerable to adversarial perturbations [1, 14]. Numerous studies demonstrate that injecting small, human-imperceptible perturbations into visual inputs can mislead vision–language models into producing incorrect or malicious outputs, exposing critical security risks [15, 34, 39]. These vulnerabilities have been observed across a variety of influential models, including CLIP [22], BLIP [14], and MiniGPT-4 [1]. Notably, adversarial attacks can remain highly effective even in black-box settings, indicating that current defense mechanisms are insufficient to address the underlying weaknesses of these models [13, 19].

However, the majority of existing adversarial attacks focus on tasks such as VQA, image–text matching, or captioning [17, 23, 27], often assuming that attackers can directly manipulate the image fed into the model. Such an assumption does not hold in GUI-agent scenarios [25]. In these settings, the model's visual input is derived from user-side screenshots that adversaries cannot directly modify. Consequently, adversarial perturbations must remain effective throughout the entire pipeline—from "user-side screenshot → preprocessing → model inference"—introducing a fundamentally different challenge compared to conventional image-based attacks.

Moreover, GUI inputs typically undergo multiple preprocessing steps (e.g., resizing, cropping, quantization), which can distort adversarial patterns and introduce gradient masking, thereby hindering optimization and significantly reducing attack transferability [2, 28]. Existing studies provide limited exploration of these pipeline-specific difficulties and lack systematic solutions tailored to GUI-agent settings.

2.2 Environment Injection Attacks

Environment Injection Attacks (EIAs) constitute a prominent threat vector within GUI-agent ecosystems [5, 8, 35]. These attacks manipulate the external environment of the agent—often by modifying webpage structures, injecting deceptive HTML elements, or crafting misleading pop-up windows—to induce erroneous decisions by the agent [16, 20, 37]. However, such approaches typically assume that the adversary possesses elevated privileges or direct access to the GUI environment, which is unrealistic in many real-world applications.

A more practical threat model considers local adversaries (e.g., end users, content creators, merchants) who can only upload localized visual assets such as advertisements, product images, or shareable thumbnails. Under these constrained capabilities, perturbations must be covertly embedded within uploaded images and remain effective even after undergoing screenshot capture, rendering, and preprocessing transformations [2, 30]. While some recent works attempt to introduce adversarial perturbations at the image level [2, 26, 28, 30, 38], these efforts largely follow conventional adversarial paradigms and often overlook the dynamic nature of GUI layouts and contextual cues.

More critically, existing approaches rarely address the transferability bottleneck introduced by the "screenshot → preprocessing → inference" pipeline, where multiple visual transformations can disrupt adversarial structures and impede gradient-based optimization [2, 28]. Although prior studies acknowledge that preprocessing operations may weaken adversarial effectiveness, they stop short of proposing a principled optimization framework capable of ensuring robustness along the full inference path. As a result, current environment injection attacks struggle to achieve both stability and high attack success rates in realistic GUI-agent scenarios.

3 Methodology

3.1 Problem Formulation and Adversary Analysis

3.1.1 Task Formulation. We focus on evaluating the robustness of MLLMs in multiple-choice question-answering (MCQA) tasks based on GUI screenshots [29, 31]. This setting simulates how GUI agents perceive interface content, identify task-relevant elements, and make interaction decisions in practical applications. It therefore provides a standardized formulation for analyzing model vulnerabilities under adversarial perturbations.

Formally, given a GUI screenshot $\mathbf{V}$, a natural-language query q, and a candidate answer set $O = \{o_1, o_2, \ldots, o_k\}$, the agent predicts an answer as:

$$\hat{o} = Agent(Pre(\mathbf{V}), q, O), \tag{1}$$

where $\mathbf{V}$ denotes the visual observation, i.e., the GUI screenshot; $Pre(\cdot)$ denotes the official preprocessing pipeline applied before feeding $\mathbf{V}$ into the MLLM; q denotes the natural-language task query; O denotes the candidate answer set; and $\hat{o} \in O$ denotes the answer predicted by the agent.

Each candidate set O contains exactly one ground-truth answer, denoted as $o^{\mathrm{gt}} \in O$. Given $(\mathbf{V}, q, O)$, the agent is expected to identify the visual information in $\mathbf{V}$ that is relevant to q, perform semantic and visual reasoning, and select the correct answer o^{gt} from O.

Under adversarial settings, we examine whether the agent remains reliable when the visual observation $\mathbf{V}$ is perturbed while the query q and candidate answer set O remain unchanged. Specifically, a successful targeted attack aims to cause the perturbed prediction to match an attacker-specified target answer $o^{\star} \in O \setminus \{o^{\mathrm{gt}}\}$. This formulation captures standard GUI understanding while establishing the basis for studying robustness under realistic threat conditions.

3.1.2 Adversary Model. Given the agent's input $(\mathbf{V}, q, O)$, where $o^{\text{gt}} \in O$ denotes the ground-truth answer, the adversary seeks to inject visually unobtrusive perturbations into the screenshot $\mathbf{V}$ such that the agent predicts an attacker-specified target answer $o^\star \in O \setminus \{o^{\text{gt}}\}$.

To reflect realistic GUI-agent deployment scenarios, we distinguish adversaries by the extent of their control over the visual interface. A *global adversary* (e.g., developers or privileged maintainers) has unrestricted access to interface layout and content, allowing perturbations on arbitrary pixels of $\mathbf{V}$. In contrast, a *local adversary* models ordinary users or merchants who can modify only specific interface regions—such as advertisement slots, product photos, or shareable thumbnails. Despite their limited privileges, such localized modifications naturally appear in user-captured screenshots and therefore constitute practical high-risk attack surfaces. Correspondingly, the perturbation mask M covers the full image under global attacks but is confined to predefined candidate regions under local attacks.

The perturbed visual observation is defined as:

$$\tilde{\mathbf{V}} = \text{clip}(\mathbf{V} + M \odot \delta,\ 0,\ 255), \tag{2}$$

where δ is the perturbation tensor and M is a binary mask encoding allowable modification regions. The adversary aims to enforce:

$$\mathit{Agent}(\text{Pre}(\tilde{\mathbf{V}}),\ q,\ O) = o^\star, \tag{3}$$

i.e., the manipulated observation induces the agent to produce a targeted incorrect prediction.

To ensure stealthiness and practical relevance, perturbations must satisfy three constraints.

(1) Magnitude and perceptual constraints. Perturbations must satisfy a norm bound, typically $\|\delta\|_\infty \le \epsilon$, and may further incorporate perceptual regularizers such as total variation (TV) to improve smoothness and imperceptibility.

(2) Access assumptions. We adopt a white-box threat model in which the adversary has full access to the agent $\mathit{Agent}(\cdot)$, including the large language model $\mathcal{G}$ with a visual encoder ϕ, and their gradients. This constitutes a standard strong-adversary setting widely used in adversarial research [2, 28].

(3) Fixed environment. To ensure clear attack modeling and reproducible experiments, several system components are considered public and immutable, including region detectors (with fixed thresholds), the preprocessing pipeline $\text{Pre}(\cdot)$ (e.g., resizing, interpolation, quantization), and decoding configurations (e.g., sampling strategy, temperature).

Optimization Objective. Under the above assumptions, the attack corresponds to solving the following constrained optimization problem:

$$\min_{\delta}\ \mathcal{L}_{\text{adv}}\left(\mathcal{G}(\phi(\text{Pre}(\text{clip}(\mathbf{V} + M \odot \delta,\ 0,\ 255))), q, O), o^\star\right) + \lambda_{\text{tv}}\mathcal{L}_{\text{tv}} \quad \text{s.t.} \quad \|\delta\|_\infty \le \epsilon. \tag{4}$$

where $\mathcal{L}_{\text{adv}}$ encourages the model to assign higher probability to the attacker-specified target answer $o^\star$ among the candidate answer set O.

3.2 Solving for the Perturbation δ

Solving Eq. 4 is nontrivial. In this section, we first analyze the intrinsic challenges of optimizing adversarial perturbations for MLLMs, and then present two complementary mechanisms designed to overcome these challenges in practical settings.

3.2.1 Challenges in Optimizing Adversarial Perturbations.

Challenge 1: Non-differentiable preprocessing. Modern MLLMs apply a sequence of preprocessing operations—including resizing, float-to-`uint8` quantization, normalization, and patchification—before forwarding inputs to the visual encoder. Several of these operations are inherently non-differentiable:

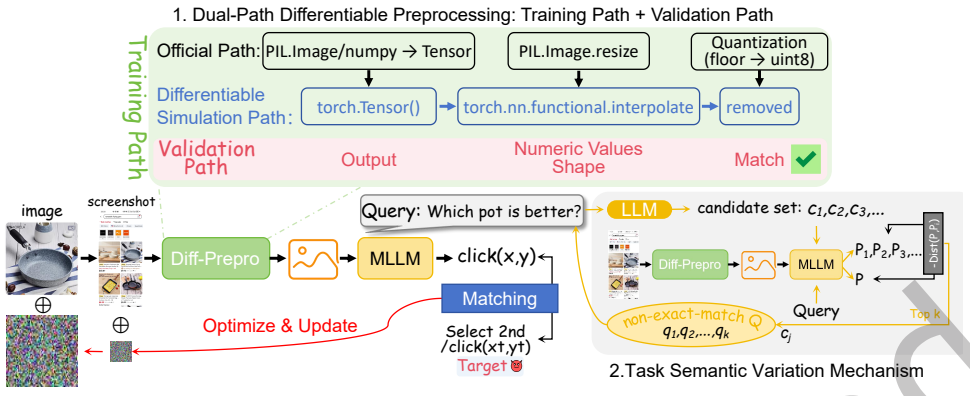

Fig. 2. The optimization framework of RAA.

- *Resizing* is commonly implemented using non-differentiable routines (e.g., `PIL.Image.resize`), blocking gradient propagation to pixel-level perturbations.
- *Quantization* yields near-zero gradients almost everywhere, further impeding backpropagation.

Consequently, gradients fail to reliably flow through the official preprocessing pipeline $Pre(\cdot)$, making perturbations optimized in the differentiable space ineffective after preprocessing is applied.

Challenge 2: Semantic overfitting. Even if differentiability is restored, optimizing perturbations with respect to a single prompt typically leads to semantic overfitting. Because MLLMs are highly sensitive to linguistic phrasing, perturbations tuned to a specific formulation of the task often fail when the query is paraphrased or semantically restructured. Minor variations—such as synonym substitution or syntactic reordering—may significantly reduce attack success.

3.2.2 Proposed Solutions. To overcome these limitations, we introduce two complementary mechanisms. First, a *dual-branch differentiable preprocessing module* restores reliable gradient flow while remaining faithful to the actual inference pipeline. Second, a *task-level semantic variation module* mitigates prompt-specific overfitting by enhancing robustness to linguistic variations.

(1) *Dual-Branch Differentiable Preprocessing.*

To handle non-differentiable operators in the preprocessing pipeline, we introduce a dual-branch architecture consisting of a differentiable branch and a validation branch:

- *Differentiable training branch.* $Pre_{\text{diff}}(\cdot)$ acts as a differentiable surrogate for the standard pipeline. It replaces non-differentiable resizing operations (e.g., PIL-based resampling) with differentiable bilinear interpolation and approximates discrete quantization (rounding to `uint8`) using continuous relaxation or straight-through estimation. This reconstruction restores the gradient flow $\nabla_\delta \mathcal{L}$, enabling effective backpropagation to the perturbation δ.
- *Validation branch.* $Pre_{\text{ref}}(\cdot)$ strictly reproduces the official preprocessing pipeline, including non-differentiable resizing and discrete quantization. During optimization, we use this branch to verify the attack's effectiveness. The optimization process terminates or updates the best-found perturbation only if the attack succeeds under the strict, non-differentiable constraints of $Pre_{\text{ref}}(\cdot)$, ensuring robustness against preprocessing artifacts.

This dual-branch design ensures reliable gradient flow through Pre_{diff} while leveraging the validation branch to guarantee that the optimized perturbations remain effective when evaluated under the true preprocessing pipeline Pre_{ref}.

(2) *Task-Level Semantic Variation.*

To mitigate semantic overfitting, we construct a set of semantically equivalent prompts that preserve the task intent while introducing controlled linguistic diversity. Formally, we define

$$\mathcal{Q} = \{q_0\} \cup \{\mathcal{T}_i(q_0)\}_{i=1}^{K-1}, \tag{5}$$

where each operator $\mathcal{T}_i$ performs a constrained paraphrasing transformation of the original prompt q_0 [9, 18].

To instantiate these transformations, we first generate a candidate set $C = \{c_j\}_{j=1}^{N}$ using a large language model [12]. We then assess the semantic alignment of each candidate by measuring the divergence

$$D_j = \text{Dist}(P_{q_0}, P_{c_j}), \tag{6}$$

where $\text{Dist}(\cdot,\cdot)$ quantifies the discrepancy between the output distributions induced by different prompts using the Jensen–Shannon divergence [24, 40]. Finally, the $K-1$ candidates with the smallest divergence scores are selected to form the final prompt set $\mathcal{Q}$, ensuring that the resulting prompts provide meaningful linguistic variation while maintaining semantic equivalence.

Final Optimization Objective. Combining both mechanisms, we formulate the final optimization problem as:

$$\min_{\delta} \frac{1}{|\mathcal{Q}|} \sum_{q\in\mathcal{Q}} \left(\mathcal{L}_{\text{adv}}\Big(\mathcal{G}(\phi(Pre_{\text{diff}}(\tilde{\mathbf{V}})),\, q,\, O),\, o^{\star}\Big) + \lambda_{\text{tv}}\, \mathcal{L}_{\text{tv}} \right) \quad \text{s.t.} \quad \|\delta\|_{\infty} \le \epsilon, \tag{7}$$

where $\tilde{\mathbf{V}} = \text{clip}(\mathbf{V} + M \odot \delta, 0, 255)$. Here, Pre_{diff} is used as a differentiable surrogate for gradient-based optimization, while Pre_{ref} is used only for validating and selecting perturbations under the actual inference pipeline.

This objective integrates differentiability constraints with semantic regularization, yielding perturbations that are effective across diverse prompt formulations and robust under realistic preprocessing pipelines. We update the perturbation δ using PGD algorithm. An overview of the optimization framework is provided in Figure 2, while the full optimization procedure is described step-by-step in Algorithm 1.

4 Experimental Evaluation

4.1 Experimental Setup

Datasets. We employ two public GUI benchmarks designed for mobile interfaces and construct a unified, reproducible evaluation set through standardized filtering and re-annotation.

ScreenSpot v2 [31]. This dataset contains large-scale mobile application screenshots with annotated interactive elements. We align screenshots with their corresponding metadata using the official annotation files, retain bounding boxes and natural-language instructions relevant to localization/selection tasks, and visualize them for analysis. The subset covers common mobile resolutions (e.g., 1080×2400, 1170×2532) and exhibits substantial diversity reflective of real-world mobile usage.

MMBench-GUI-Mobile [29]. Derived from the secondary annotation subset of MMBench-GUI (focused on element localization/selection), this dataset includes mobile screenshots paired with interaction-element metadata. We retain Android and iOS entries for consistency and prioritize samples containing multiple candidate interactive regions to form a more challenging multi-target localization subset. As with ScreenSpot v2, this dataset is organized as "bounding box + natural-language instruction'' pairs and covers typical mobile resolutions (e.g., 1080×2340, 1179×2556).

Algorithm 1: Robust Adversarial Attack (RAA)

Input: Agent with base model $\mathcal{G}$ and visual encoder ϕ; clean image $\mathbf{V}$; original query q_0; answer set O; target answer $o^\star$; perturbation mask M; maximum perturbation budget ϵ; step size α; number of PGD steps T; differentiable preprocessing Pre_{diff}; reference preprocessing Pre_{ref}; TV weight λ_{tv}; prompt candidate pool size N; final prompt number K.

Output: Adversarial perturbation δ_{best}.

Generate a candidate set $C = c_j{}_{j=1}^{N}$ from q_0 using an LLM under a non-exact-match constraint;
$P_{q_0} \leftarrow \mathcal{G}\left(\phi\left(Pre_{\text{diff}}(\mathbf{V})\right), q_0, O\right)$;
for $j = 1$ **to** N **do**
    $P_{c_j} \leftarrow \mathcal{G}\left(\phi\left(Pre_{\text{diff}}(\mathbf{V})\right), c_j, O\right)$;
    $D_j \leftarrow \text{JSD}(P_{q_0}, P_{c_j})$;
Let C_{top} be the first $(K-1)$ elements of C sorted in ascending order of D_j;
$Q \leftarrow q_0 \cup C_{\text{top}}$;
Initialize $\delta \leftarrow \mathbf{0}$;
Initialize $\delta_{\text{best}} \leftarrow \delta$;
Initialize $s_{\text{best}} \leftarrow -1$; `// Best real-pipeline success count`
Initialize $\mathcal{L}_{\text{best}} \leftarrow +\infty$; `// Tie-breaker when success counts are equal`
for $t = 1$ **to** T **do**
    $\tilde{\mathbf{V}} \leftarrow \text{clip}(\mathbf{V} + M \odot \delta, 0, 255)$;
    $\mathbf{Z}_{\text{diff}} \leftarrow Pre_{\text{diff}}(\tilde{\mathbf{V}})$; `// Differentiable branch`
    $\mathcal{L}_{\text{adv}} \leftarrow 0$;
    foreach $q \in Q$ **do**
        $\mathcal{L}_{\text{adv}} \leftarrow \mathcal{L}_{\text{adv}} + \mathcal{L}_{\text{CE}}\left(\mathcal{G}\left(\phi(\mathbf{Z}_{\text{diff}}), q, O\right), o^\star\right)$;
    $\mathcal{L}_{\text{adv}} \leftarrow \mathcal{L}_{\text{adv}} / |Q|$;
    $\mathcal{L}_{\text{tv}} \leftarrow \text{TV}(M \odot \delta)$;
    $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{adv}} + \lambda_{\text{tv}} \mathcal{L}_{\text{tv}}$;
    $\delta \leftarrow \delta - \alpha \cdot \text{sign}\left(\nabla_\delta \mathcal{L}_{\text{total}}\right)$; `// Targeted PGD update`
    $\delta \leftarrow M \odot \text{clip}(\delta, -\epsilon, \epsilon)$; `// Masked projection`
    if $((t \bmod 50) = 0) \vee (t = T)$ **then**
        $\tilde{\mathbf{V}}_{\text{ref}} \leftarrow \text{clip}(\mathbf{V} + M \odot \delta, 0, 255)$;
        $\mathbf{Z}_{\text{ref}} \leftarrow Pre_{\text{ref}}(\tilde{\mathbf{V}}_{\text{ref}})$; `// Reference branch`
        $s \leftarrow 0$;
        foreach $q \in Q$ **do**
            $\hat{o}_{\text{real}} \leftarrow Agent(\mathbf{Z}_{\text{ref}}, q, O)$;
            if $\hat{o}_{\text{real}} = o^\star$ **then**
                $s \leftarrow s + 1$;
        if $(s > s_{\text{best}}) \vee ((s = s_{\text{best}}) \wedge (\mathcal{L}_{\text{adv}} < \mathcal{L}_{\text{best}}))$ **then**
            $s_{\text{best}} \leftarrow s$;
            $\mathcal{L}_{\text{best}} \leftarrow \mathcal{L}_{\text{adv}}$;
            $\delta_{\text{best}} \leftarrow \delta$;
        if $s = |Q|$ **then**
            break;
return δ_{best};

Table 1. Statistics of the two mobile GUI datasets used in our experiments. We report per-platform image and annotation counts, instruction counts, and the average (variance) of options per sample.

Dataset	Platform	Images	Annotations	Instructions	Options
ScreenSpot-Mobile v2	Android	86	211	199	2.45 (0.43)
ScreenSpot-Mobile v2	iOS	110	238	227	2.16 (1.46)
MMBench-GUI-Mobile	Android	39	88	88	2.26 (0.24)
MMBench-GUI-Mobile	iOS	70	288	288	4.11 (3.76)

Dataset statistics are shown in Table 1. *Images* denotes the number of screenshots, *Annotations* and *Instructions* correspond to element annotations and natural-language task prompts, and *Options* reports the average (variance) number of candidate regions per screenshot.

Models. We evaluate two representative categories of GUI grounding models as attack targets. To ensure fairness and comparability, all models are tested using their official open-source full-precision weights and the corresponding recommended inference pipelines.

Qwen2.5-VL-3B/7B-Instruct. A general-purpose vision–language model from the Qwen family [4], equipped with strong cross-modal understanding and instruction-following capabilities.

GUI-Owl-7B. Developed as part of the MobileAgent project [33], this model extends Qwen2.5-VL with large-scale GUI-specific interaction data, followed by trajectory optimization and reinforcement learning. These enhancements significantly improve its interface-element understanding, long-horizon task planning, and interaction stability.

Evaluation Metrics. We evaluate model performance under both clean and adversarial inputs using the following metrics:

- *ACC (Accuracy)*: Measures the agreement between the model's prediction and the ground-truth answer.
- *ASR (Attack Success Rate)*: Measures the proportion of cases in which the model outputs the attacker-specified target while deviating from the ground truth, reflecting the effectiveness of targeted attacks.

Baselines and Defenses. We compare our method against two representative attack strategies: *Popup Attack* [37]: inserts fake pop-ups or overlays into screenshots to mislead models in a black-box setting; *Targeted Attack* [38]: a gradient-based pixel-level attack that applies PGD to craft perturbations within a specified budget, but operates on the *post-preprocessed* visual input. Because preprocessing in models like Qwen2VL destroys spatial locality through patchification and reordering, Targeted Attack cannot restrict perturbations to any subregion of the image and only supports global perturbations.

We further evaluate each attack under three lightweight input-preprocessing defenses: *Gaussian Noise* [7, 36]: adding zero-mean Gaussian noise ($\sigma \in \{2, 5\}$) followed by clipping to $[0, 255]$; *JPEG Compression* [11]: applying lossy JPEG compression with quality factors $q \in \{95, 75\}$; *Resize* [32]: reducing images to 90% or 75% of the original size and restoring via Lanczos interpolation to test sensitivity to resolution changes.

Key Configurations. For reproducibility and clarity, we summarize the key configurations below.

Data and Preprocessing. We construct two complementary mobile subsets (`ScreenSpot v2` and `MMBench-GUI-Mobile`), where each sample retains the "screenshot–bounding-box–instruction" structure. Candidate attack regions are automatically generated following fixed ratio–position rules (top, center, bottom; proportional square or random square), covering either full-image regions or partial regions (e.g., 10%, 30%, . . . , 100%). All inputs are uniformly downscaled by 0.45 prior to model-aligned preprocessing.

Optimization Settings. Adversarial perturbations are optimized using a projected gradient descent (PGD) procedure under an ℓ_∞ constraint. The perturbation budget is set to $\epsilon = 16$, each PGD update applies a step

size of 1, and the optimization is performed for a total of 300 iterations. After every update, the perturbation is projected onto the feasible set defined by the mask support and the ℓ_∞ bound to ensure constraint satisfaction. During optimization, every 50 iterations we evaluate the current perturbation on an `eval_prompts` set; if all prompts satisfy the attacker-specified output, the optimization terminates early and the current best perturbation is retained.

Unless otherwise specified, all experiments adopt the default configurations above, with the attack region placed at the bottom of the interface. For inference, we use deterministic decoding (temperature 0) to guarantee reproducibility. All experiments are executed on NVIDIA A100 GPUs (8×80 GB).

4.2 Main Results

4.2.1 ***RAA Consistently Outperforms Existing Attacks Across Models and Datasets.*** Table 2 reports the performance of three attack methods—Popup Attack, Targeted Attack, and our proposed RAA—on the MMBench-GUI and ScreenSpot-V2 datasets using multiple MLLM-based GUI agents. Experiments are conducted under two perturbation-region settings (Region = 0.1 and 1.0), corresponding to localized and global attacks, respectively. Due to its reliance on optimizing the "after data preprocessing → model inference" path, Targeted Attack becomes non-differentiable under Region = 0.1 in our threat model; thus, its results are marked as "–". This highlights a fundamental incompatibility of Targeted Attack with real GUI-Agent pipelines. Overall, **RAA achieves the highest or near-highest ASR across all evaluated model–dataset pairs**, accompanied by consistently larger reductions in ACC. We further note that the fine-tuned GUI-Owl model does not outperform Qwen2.5-VL-3B or Qwen2.5-VL-7B on our evaluated tasks due to task mismatch.

Across MMBench-GUI, RAA consistently delivers the highest ASR, even when restricted to minimal perturbation regions. On GUI-Owl, Popup Attack yields 61.0%/69.7% ASR under Region = 0.1/1.0, while Targeted Attack reaches 72.5% under Region = 1.0. In contrast, RAA achieves 87.2% and 98.9% under the same settings and already surpasses both baselines under Region = 0.1. A similar pattern holds for Qwen2.5-VL-3B: Popup Attack obtains 65.8%/83.0%; Targeted Attack reaches 95.4% under Region = 1.0; yet RAA attains 92.7% at Region = 0.1 and 98.2% at Region = 1.0. For Qwen2.5-VL-7B, the gap becomes more pronounced: Popup Attack reaches 45.9% under Region = 1.0, while RAA achieves 76.6% under Region = 0.1 and 100% under Region = 1.0. These results demonstrate that RAA remains highly effective even under extremely limited perturbation budgets.

On ScreenSpot-V2, RAA maintains its superiority despite the dataset's more complex and diverse UI layouts. ScreenSpot-V2 reflects more realistic mobile UI variability, yet RAA maintains its advantage. On GUI-Owl, Popup Attack reaches 45.0% under Region = 1.0, and Targeted Attack reaches 76.3%. However, RAA achieves 76.6% under Region = 0.1 and 95.8% under Region = 1.0, outperforming all baselines even when they operate with full-image perturbations. For Qwen2.5-VL-7B, Popup Attack and Targeted Attack achieve 48.0% and 95.3% at Region = 1.0, whereas RAA achieves 51.8% under Region = 0.1 and 100% under Region = 1.0. These results indicate that RAA retains high attack transferability across models and remains effective under complex, real-world UI structures.

Model scale strongly influences vulnerability, with smaller models more susceptible overall, yet even large models ultimately fail under RAA. Model scale significantly affects vulnerability: smaller models consistently exhibit higher ASR under the same perturbation budget, likely due to weaker visual encoding and cross-modal alignment. This trend is particularly evident under Region = 0.1 for both datasets, where Qwen2.5-VL-3B is more vulnerable than Qwen2.5-VL-7B. Crucially, however, even the larger and more capable Qwen2.5-VL-7B completely collapses under RAA's global perturbation (ASR = 100%). These findings reinforce that simply scaling model capacity is insufficient to secure GUI-Agent systems.

Table 2. Attack performance across datasets, models, and perturbation ratios. Metrics include accuracy under the corresponding input condition (ACC) and targeted attack success rate (ASR).

Dataset	Model	Attack Method	Region	ACC	ASR
MMBench-GUI	GUI-Owl	No Attack	-	44.2	-
MMBench-GUI	GUI-Owl	Popup Attack	0.1	$26.8_{(\downarrow 17.4)}$	61.0
MMBench-GUI	GUI-Owl	Popup Attack	1.0	$16.7_{(\downarrow 27.5)}$	69.7
MMBench-GUI	GUI-Owl	Targeted Attack	0.1	-	-
MMBench-GUI	GUI-Owl	Targeted Attack	1.0	$16.3_{(\downarrow 27.9)}$	72.5
MMBench-GUI	GUI-Owl	RAA(ours)	0.1	$7.6_{(\downarrow 36.6)}$	87.2
MMBench-GUI	GUI-Owl	RAA(ours)	1.0	$0.0_{(\downarrow 44.2)}$	98.9
MMBench-GUI	Qwen2.5-VL-3B	No Attack	-	69.1	-
MMBench-GUI	Qwen2.5-VL-3B	Popup Attack	0.1	$31.9_{(\downarrow 37.2)}$	65.8
MMBench-GUI	Qwen2.5-VL-3B	Popup Attack	1.0	$14.2_{(\downarrow 54.9)}$	83.0
MMBench-GUI	Qwen2.5-VL-3B	Targeted Attack	0.1	-	-
MMBench-GUI	Qwen2.5-VL-3B	Targeted Attack	1.0	$2.3_{(\downarrow 66.8)}$	95.4
MMBench-GUI	Qwen2.5-VL-3B	RAA(ours)	0.1	$5.5_{(\downarrow 63.6)}$	92.7
MMBench-GUI	Qwen2.5-VL-3B	RAA(ours)	1.0	$0.0_{(\downarrow 69.1)}$	98.2
MMBench-GUI	Qwen2.5-VL-7B	No Attack	-	87.8	-
MMBench-GUI	Qwen2.5-VL-7B	Popup Attack	0.1	$59.2_{(\downarrow 28.7)}$	36.7
MMBench-GUI	Qwen2.5-VL-7B	Popup Attack	1.0	$47.9_{(\downarrow 39.9)}$	45.9
MMBench-GUI	Qwen2.5-VL-7B	Targeted Attack	0.1	-	-
MMBench-GUI	Qwen2.5-VL-7B	Targeted Attack	1.0	$3.2_{(\downarrow 84.6)}$	93.6
MMBench-GUI	Qwen2.5-VL-7B	RAA(ours)	0.1	$19.0_{(\downarrow 68.8)}$	76.6
MMBench-GUI	Qwen2.5-VL-7B	RAA(ours)	1.0	$0.0_{(\downarrow 87.8)}$	100.0
ScreenSpot-V2	GUI-Owl	No Attack	-	60.3	-
ScreenSpot-V2	GUI-Owl	Popup Attack	0.1	$34.6_{(\downarrow 25.7)}$	46.3
ScreenSpot-V2	GUI-Owl	Popup Attack	1.0	$34.6_{(\downarrow 25.6)}$	45.0
ScreenSpot-V2	GUI-Owl	Targeted Attack	0.1	-	-
ScreenSpot-V2	GUI-Owl	Targeted Attack	1.0	$14.7_{(\downarrow 45.6)}$	76.3
ScreenSpot-V2	GUI-Owl	RAA(ours)	0.1	$12.3_{(\downarrow 48.0)}$	76.6
ScreenSpot-V2	GUI-Owl	RAA(ours)	1.0	$1.4_{(\downarrow 58.9)}$	95.8
ScreenSpot-V2	Qwen2.5-VL-3B	No Attack	-	89.7	-
ScreenSpot-V2	Qwen2.5-VL-3B	Popup Attack	0.1	$55.8_{(\downarrow 33.9)}$	38.6
ScreenSpot-V2	Qwen2.5-VL-3B	Popup Attack	1.0	$38.2_{(\downarrow 51.4)}$	51.0
ScreenSpot-V2	Qwen2.5-VL-3B	Targeted Attack	0.1	-	-
ScreenSpot-V2	Qwen2.5-VL-3B	Targeted Attack	1.0	$3.8_{(\downarrow 85.9)}$	94.8
ScreenSpot-V2	Qwen2.5-VL-3B	RAA(ours)	0.1	$25.6_{(\downarrow 64.1)}$	72.9
ScreenSpot-V2	Qwen2.5-VL-3B	RAA(ours)	1.0	$0.6_{(\downarrow 89.1)}$	99.4
ScreenSpot-V2	Qwen2.5-VL-7B	No Attack	-	93.9	-
ScreenSpot-V2	Qwen2.5-VL-7B	Popup Attack	0.1	$67.7_{(\downarrow 26.2)}$	29.2
ScreenSpot-V2	Qwen2.5-VL-7B	Popup Attack	1.0	$44.1_{(\downarrow 49.8)}$	48.0
ScreenSpot-V2	Qwen2.5-VL-7B	Targeted Attack	0.1	-	-
ScreenSpot-V2	Qwen2.5-VL-7B	Targeted Attack	1.0	$4.7_{(\downarrow 89.2)}$	95.3
ScreenSpot-V2	Qwen2.5-VL-7B	RAA(ours)	0.1	$44.5_{(\downarrow 49.4)}$	51.8
ScreenSpot-V2	Qwen2.5-VL-7B	RAA(ours)	1.0	$0.0_{(\downarrow 93.9)}$	100.0

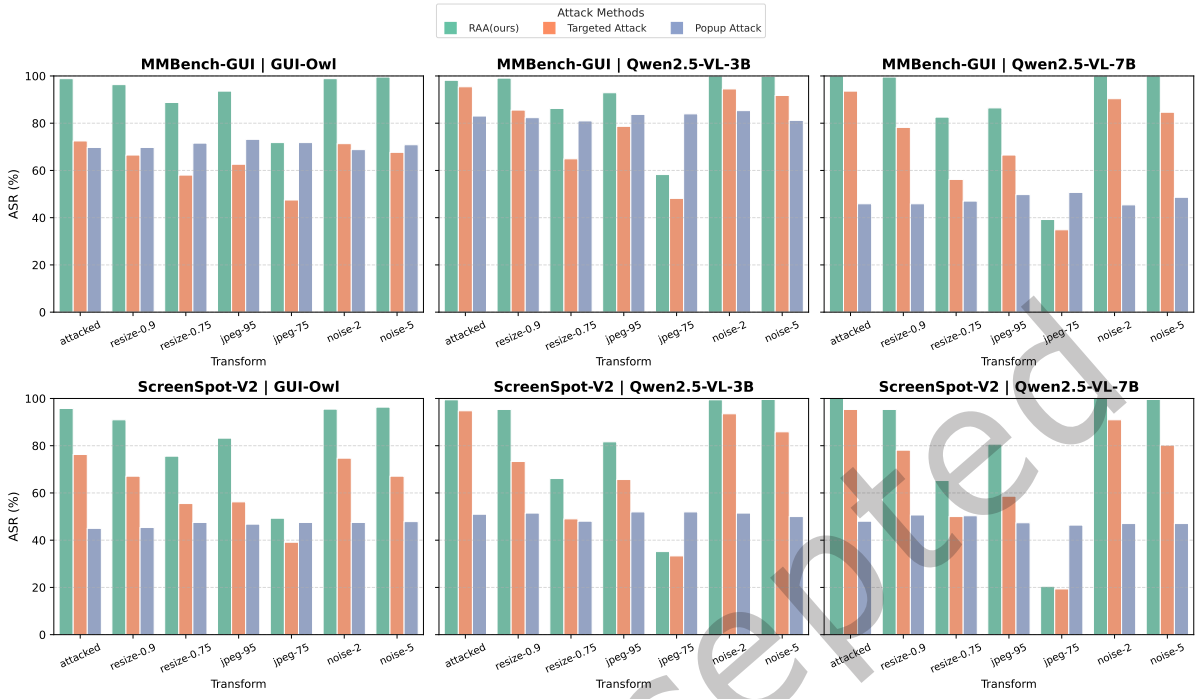

Fig. 3. Robustness comparison of three adversarial attack methods under typical image-level defense transformations across multiple datasets and model architectures at perturbation Region = 1.0.

In summary, RAA exhibits strong generalizability across models, datasets, and perturbation budgets, and its attack performance consistently exceeds that of existing methods. The results highlight RAA as a practically realizable and impactful threat to current GUI-Agent systems, underscoring the need for robust defense mechanisms.

4.2.2 ***RAA Remains Highly Effective Under Common Image-Level Defenses, Demonstrating Superior Stability and Robustness.*** As shown in Fig. 3, we systematically compare the attack success rates of the three attack methods under several common image-level defenses, including resizing, JPEG compression, and Gaussian noise. Overall, as the defense strength increases (e.g., smaller resize factors, lower JPEG quality, or larger noise magnitude), the ASR of all methods decreases to varying extents; however, their sensitivity and degradation patterns differ substantially.

RAA consistently achieves the highest or near-highest ASR across all models and both datasets. Although stronger defenses (e.g., `resize-0.75`, `jpeg-75`, `noise-5`) inevitably reduce overall ASR, the degradation of RAA is markedly smaller than that of Targeted Attack. Across all MMBench-GUI and ScreenSpot-V2 models, RAA maintains a clear advantage under `jpeg-75` and `noise-5`, indicating that its perturbations exhibit stronger resilience to compression artifacts and random noise.

Popup Attack shows relatively high stability against defenses—its ASR varies only minimally across defense configurations, particularly under strong resizing and compression. This behavior arises because Popup Attack injects explicit textual overlays rather than fine-grained pixel perturbations; such high-level text regions are largely preserved under image-level transformations, making the attack less susceptible to disruption. However,

Table 3. Effect of task-level semantic variation on RAA performance under two perturbation ratios. We report ASR and average optimization time for Region = 0.1 and Region = 1.0 under different numbers of semantic variants $K = 1, 2, 3, 4$.

Dataset	Model	Semantic Variation	Region = 0.1		Region = 1.0	
			ASR	Avg Time (s)	ASR	Avg Time (s)
MMBench-GUI	GUI-Owl	K=1	68.6	78.9	78.9	65.0
MMBench-GUI	GUI-Owl	K=2	75.9	100.4	84.2	69.2
MMBench-GUI	GUI-Owl	K=3	78.0	108.9	97.0	88.8
MMBench-GUI	GUI-Owl	K=4	87.2	138.1	98.9	99.3
MMBench-GUI	Qwen2.5-VL-3B	K=1	83.9	131.3	99.5	108.3
MMBench-GUI	Qwen2.5-VL-3B	K=2	84.2	247.5	99.8	251.4
MMBench-GUI	Qwen2.5-VL-3B	K=3	88.3	326.2	99.1	285.8
MMBench-GUI	Qwen2.5-VL-3B	K=4	92.7	326.5	98.2	308.8
MMBench-GUI	Qwen2.5-VL-7B	K=1	64.7	157.7	99.5	162.2
MMBench-GUI	Qwen2.5-VL-7B	K=2	66.3	188.3	98.6	208.3
MMBench-GUI	Qwen2.5-VL-7B	K=3	71.3	211.1	100.0	208.4
MMBench-GUI	Qwen2.5-VL-7B	K=4	76.6	217.5	100.0	209.9
ScreenSpot-V2	GUI-Owl	K=1	52.7	76.0	71.0	56.8
ScreenSpot-V2	GUI-Owl	K=2	67.1	123.1	83.8	84.5
ScreenSpot-V2	GUI-Owl	K=3	73.7	193.7	94.1	121.7
ScreenSpot-V2	GUI-Owl	K=4	76.6	195.5	95.8	123.1
ScreenSpot-V2	Qwen2.5-VL-3B	K=1	48.2	126.5	99.3	76.4
ScreenSpot-V2	Qwen2.5-VL-3B	K=2	65.7	385.8	99.9	265.8
ScreenSpot-V2	Qwen2.5-VL-3B	K=3	66.1	394.9	100.0	362.9
ScreenSpot-V2	Qwen2.5-VL-3B	K=4	72.9	406.8	99.4	365.5
ScreenSpot-V2	Qwen2.5-VL-7B	K=1	37.0	197.8	99.0	162.4
ScreenSpot-V2	Qwen2.5-VL-7B	K=2	39.8	218.9	99.2	225.9
ScreenSpot-V2	Qwen2.5-VL-7B	K=3	44.9	251.0	99.9	233.8
ScreenSpot-V2	Qwen2.5-VL-7B	K=4	51.8	258.2	100.0	248.5

due to its inherently weak attack capability, Popup Attack consistently underperforms both RAA and Targeted Attack in absolute ASR. Its apparent robustness therefore reflects the preservation of these textual overlays rather than genuine resilience to defensive transformations.

In contrast, *Targeted Attack experiences substantial performance degradation under all defenses, with drops significantly larger than those of RAA*. On models such as Qwen2.5-VL-3B and Qwen2.5-VL-7B, both `resize-0.75` and `jpeg-75` cause Targeted Attack to lose up to 30%–40% of its original ASR. Under strong JPEG compression, its performance often approaches—or even falls below—that of Popup Attack. These observations highlight the method's sensitivity to pixel-level distortions and its lack of robustness to realistic input transformations.

In summary, RAA demonstrates substantially higher robustness across model architectures, datasets, and defense strategies. It consistently maintains the highest ASR under diverse perturbation transformations and exhibits a much smoother degradation curve as defenses intensify. This robustness suggests that RAA poses the most realistic and impactful threat among the evaluated attack methods, underscoring the urgency of developing stronger defenses for deployed GUI-Agent systems.

4.3 Ablation Study

4.3.1 ***Semantic Variation Generally Enhances RAA but Exhibits Strong Model- and Scenario-Dependent Behavior, with Optimization Cost Increasing with*** K**.** Table 3 summarizes the effect of task-level semantic variation on RAA under varying numbers of semantic task variants $K \in \{1, 2, 3, 4\}$ and two perturbation-region settings (Region = 0.1 and 1.0). We report attack success rate (ASR) and average optimization time to characterize both attack effectiveness and computational cost. Overall, increasing semantic variation tends to improve ASR, but the improvements are neither strictly monotonic nor consistent across models and datasets, indicating that semantic diversity interacts closely with model architecture, semantic understanding capability, and perturbation region size.

For GUI-Owl, semantic variation yields consistent and substantial monotonic improvements across datasets and perturbation-region settings. On MMBench-GUI, ASR under Region = 0.1 increases steadily from 68.6% ($K = 1$) to 87.2% ($K = 4$), while Region = 1.0 improves from 78.9% to 98.9. ScreenSpot-V2 exhibits the same pattern: ASR grows from 52.7% to 76.6% under Region = 0.1, and from 71.0% to 95.8% under Region = 1.0. These results indicate that GUI-Owl is highly sensitive to semantic perturbations, and semantic diversification strengthens attack reachability in both local and global perturbation scenarios. The optimization time grows approximately linearly in K, demonstrating the expected computational overhead of semantic augmentation.

For Qwen2.5-VL-3B, ASR remains high overall, with semantic variation providing moderate yet diminishing gains. On MMBench-GUI, Region = 0.1 yields a modest increase from 83.9% to 92.7%, while Region = 1.0 remains consistently near 100% for all K, indicating minimal marginal benefit in the global-perturbation setting. On ScreenSpot-V2, Region = 0.1 sees ASR grow from 48.2% to 72.9%, but Region = 1.0 again remains at 99.3%–100.0% across all configurations. Overall, Qwen2.5-VL-3B gains some benefit from semantic variation, yet its semantic capacity appears sufficiently strong that additional variants provide limited incremental improvement. Optimization cost increases steadily with K, with ScreenSpot-V2 showing particularly high overhead.

For Qwen2.5-VL-7B, semantic variation produces only limited and non-monotonic improvements, revealing strong robustness in semantic space. On MMBench-GUI, Region = 0.1 improves from 64.7% ($K = 1$) to 76.6% ($K = 4$), though the gain is smaller than that observed in GUI-Owl or Qwen2.5-VL-3B. Region = 1.0 remains nearly saturated (99.5%–100.0%) for all K, leaving little room for further improvement. On ScreenSpot-V2, Region = 0.1 fluctuates within the range 37.0%–51.8%, showing non-monotonic behavior, while Region = 1.0 again approaches 100% for all K. These findings indicate that Qwen2.5-VL-7B exhibits strong robustness in semantic space, making it difficult for semantic diversification alone to enhance attack performance; its behavior is governed more by dataset characteristics and perturbation region size than by the number of semantic variants.

In summary, semantic variation is a lightweight but effective attack-enhancement mechanism, particularly for smaller models, while larger models exhibit limited and inconsistent benefits. Its effectiveness strongly depends on model capacity, dataset characteristics, and perturbation region size, and it incurs predictable linear computational cost. Consequently, semantic augmentation is best used as an auxiliary strategy whose optimal configuration should consider both model scale and available attack resources.

4.3.2 ***Perturbation Region Size Exerts a Systematic and Highly Consistent Influence on Attack Success Across Models and Datasets.*** Figure 4 reports the ASR of multiple models on both datasets under different perturbation region ratios (10%, 50%, 100%). The results reveal a clear and universal trend: ASR increases monotonically and substantially as the perturbation region expands. This pattern holds across all datasets, tasks, and model sizes, indicating that the available perturbation space is a primary determinant of attack effectiveness, independent of architectural or semantic differences.

Model-level behaviors further reinforce this conclusion. Under small perturbation regions, models exhibit notable ASR variability—particularly Qwen2.5-VL-7B, whose ASR at 10% coverage is significantly lower than that of smaller models. However, as the region grows to 50% and eventually 100%, these disparities diminish, and

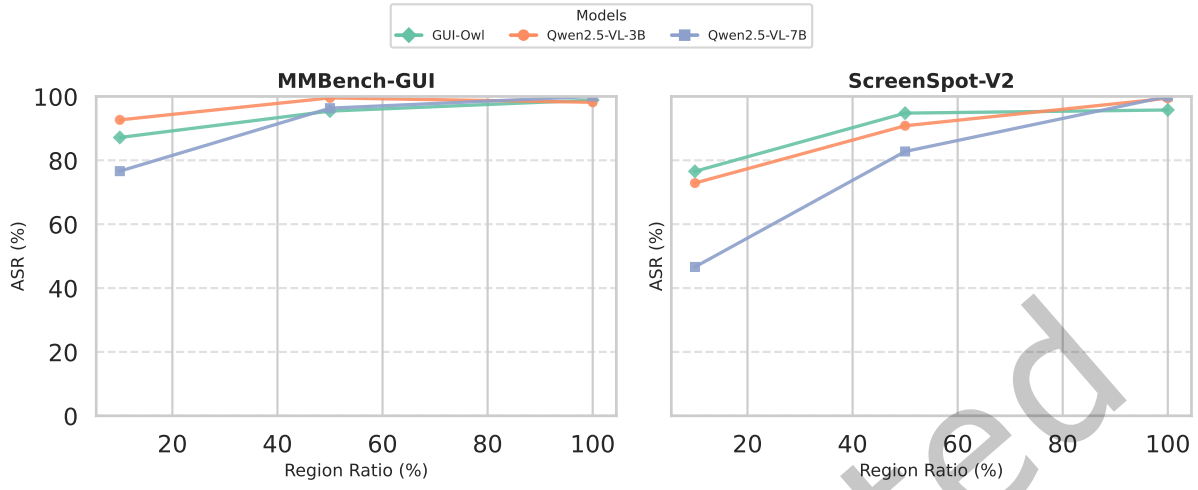

Fig. 4. Attack Success Rate (ASR) of different models under varying perturbation region ratios. Larger regions allow stronger adversarial influence, while smaller regions highlight robustness differences.

model performance converges. This suggests that larger models exhibit greater robustness to highly localized perturbations but remain comparably vulnerable when the perturbation spans the full spatial extent. In other words, model scale provides only limited protection under small regions and fails to offer meaningful robustness when perturbations become global.

Under RAA, region scaling reveals clear model-dependent robustness differences. RAA consistently benefits from larger perturbation regions, indicating that the optimization process can exploit additional spatial degrees of freedom to construct more effective adversarial patterns. Although larger models such as Qwen2.5-VL-7B show stronger robustness under highly localized perturbations, this advantage diminishes as the perturbation region expands. Taken together, perturbation region size is a dominant factor shaping attack success across models and datasets.

4.3.3 ***The Benefits of Larger Perturbation Regions Persist Under Diverse Image-Level Defenses, Demonstrating Cross-Defense Generality.*** Figure 5 examines the impact of perturbation region size under several common image-level defenses, including resizing, JPEG compression, and Gaussian noise. While these defenses uniformly reduce attack success to varying degrees, the core phenomenon remains unchanged: across all models, defenses, and perturbation strengths, expanding the perturbation region consistently improves ASR. This consistency demonstrates that region size offers a form of robustness that is defense-agnostic and arises from the spatial persistence and redundancy of large-area perturbations.

The structural reasons underlying this generality differ across defenses: (1) *Resizing*: localized perturbations are easily blurred or suppressed during downsampling, whereas larger regions preserve more structural content across scales, improving perturbation survivability. (2) *JPEG compression*: compression aggressively attenuates high-frequency signals; thus, localized fine-grained perturbations degrade quickly, while global perturbations with stronger low-frequency components remain destructive after compression. (3) *Gaussian noise*: local perturbations are more easily masked by noise, whereas larger regions benefit from spatial redundancy and cross-pixel reinforcement, enabling them to maintain attack effectiveness.

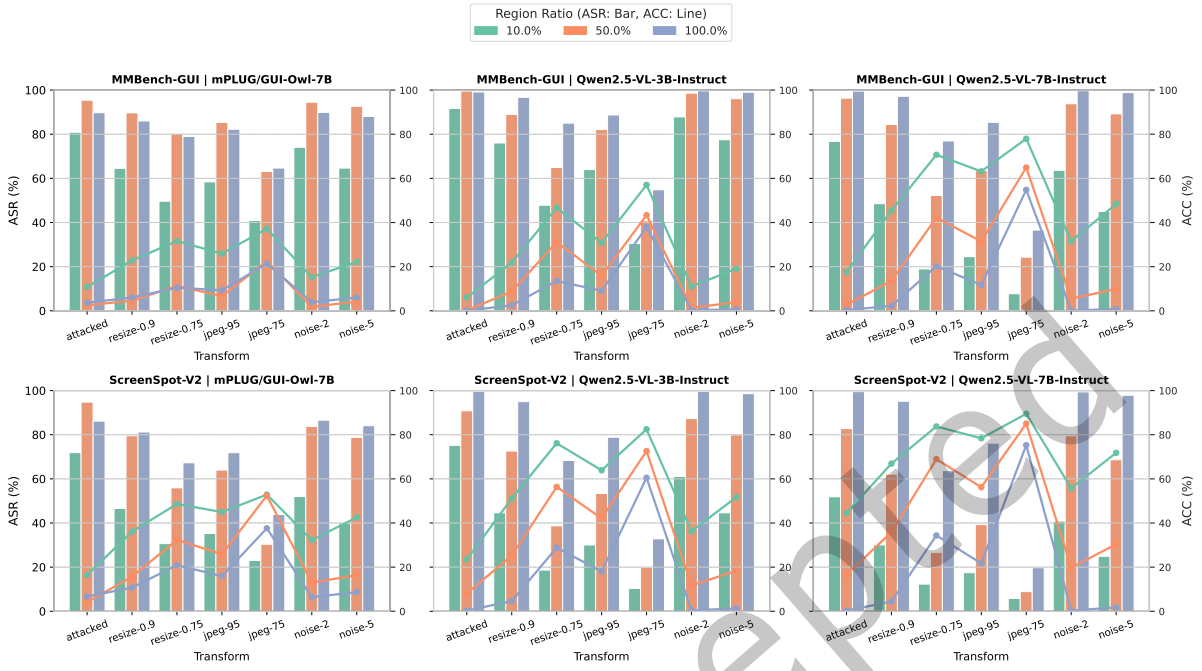

Fig. 5. Robustness analysis of adversarial examples under different defense settings. The subplots illustrate how Attack Success Rate (ASR) and task Accuracy (ACC) vary as defenses are applied. Higher ACC and lower ASR indicate stronger robustness.

Overall, the findings show that perturbation region size remains a reliable and influential factor even in the presence of image-level defenses. Enlarging the region counteracts the distortions introduced by resizing, compression, and noise, allowing the attack to retain high effectiveness under diverse input transformations. This highlights region size as a key parameter for understanding adversarial robustness in realistic GUI-Agent pipelines and underscores its importance in evaluating and securing deployed systems.

5 Conclusion

This work revisits the security of GUI agent systems built upon modern MLLMs and exposes a critical mismatch between existing adversarial attack methodologies and realistic deployment conditions. By explicitly modeling the full interaction pipeline—from client-side screenshot acquisition to model inference—we demonstrate that many prior attacks implicitly assume infeasible adversarial capabilities and fail once standard preprocessing operations are applied. To address this gap, we develop a practically realizable adversarial attack framework that integrates a dual-branch differentiable preprocessing mechanism—restoring end-to-end gradient flow while preserving fidelity to the official inference process—together with a task-level semantic variation strategy that suppresses instruction-specific overfitting and enhances robustness to linguistic variation. Our findings underscore that GUI agents remain vulnerable even under realistic threat constraints, and that perturbation region, and semantic diversity play decisive roles in attack success. We hope this work provides a foundation for future research on GUI-agent robustness, including defensive preprocessing strategies, adversarially robust training and secure interface design.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Lukas Aichberger, Alasdair Paren, Yarin Gal, Philip Torr, and Adel Bibi. 2025. Attacking multimodal os agents with malicious image patches. *arXiv preprint arXiv:2503.10809* (2025).

[3] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. 2025. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631* (2025).

[4] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).

[5] Tri Cao, Bennett Lim, Yue Liu, Yuan Sui, Yuexin Li, Shumin Deng, Lin Lu, Nay Oo, Shuicheng Yan, and Bryan Hooi. 2025. VPI-Bench: Visual Prompt Injection Attacks for Computer-Use Agents. *arXiv preprint arXiv:2506.02456* (2025).

[6] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. 2024. Seeclick: Harnessing gui grounding for advanced visual gui agents. *arXiv preprint arXiv:2401.10935* (2024).

[7] Adam Dziedzic and Sanjay Krishnan. [n. d.]. A Perturbation Analysis of Input Transformations for Adversarial Attacks. ([n. d.]).

[8] Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. 2025. Wasp: Benchmarking web agent security against prompt injection attacks. *arXiv preprint arXiv:2504.18575* (2025).

[9] Wee Chung Gan and Hwee Tou Ng. 2019. Improving the robustness of question answering systems to question paraphrasing. In *Proceedings of the 57th annual meeting of the association for computational linguistics*. 6065–6075.

[10] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=kxnoqaisCT

[11] Chuan Guo, Mayank Rana, Moustapha Cisse, and Laurens Van Der Maaten. 2017. Countering adversarial images using input transformations. *arXiv preprint arXiv:1711.00117* (2017).

[12] Or Honovich, Uri Shaham, Samuel Bowman, and Omer Levy. 2023. Instruction induction: From few examples to natural language task descriptions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1935–1952.

[13] Yilei Jiang, Xinyan Gao, Tianshuo Peng, Yingshui Tan, Xiaoyong Zhu, Bo Zheng, and Xiangyu Yue. 2025. Hiddendetect: Detecting jailbreak attacks against large vision-language models via monitoring hidden states. *arXiv preprint arXiv:2502.14744* (2025).

[14] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*. PMLR, 19730–19742.

[15] Zongxia Li, Xiyang Wu, Hongyang Du, Fuxiao Liu, Huy Nghiem, and Guangyao Shi. 2025. A Survey of State of the Art Large Vision Language Models: Alignment, Benchmark, Evaluations and Challenges. arXiv:2501.02189 [cs.CV] https://arxiv.org/abs/2501.02189

[16] Zeyi Liao, Lingbo Mo, Chejian Xu, Mintong Kang, Jiawei Zhang, Chaowei Xiao, Yuan Tian, Bo Li, and Huan Sun. [n. d.]. EIA: ENVIRONMENTAL INJECTION ATTACK ON GENERALIST WEB AGENTS FOR PRIVACY LEAKAGE. In *The Thirteenth International Conference on Learning Representations*.

[17] Li Liu, Diji Yang, Sijia Zhong, Kalyana Suma Sree Tholeti, Lei Ding, Yi Zhang, and Leilani Gilpin. 2024. Right this way: Can VLMs Guide Us to See More to Answer Questions? *Advances in Neural Information Processing Systems* 37 (2024), 132946–132976.

[18] Qin Liu, Fei Wang, Nan Xu, Tianyi Lorena Yan, Tao Meng, and Muhao Chen. 2024. Monotonic paraphrasing improves generalization of language model prompting. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 9861–9877.

[19] Weikai Lu, Hao Peng, Huiping Zhuang, Cen Chen, and Ziqian Zeng. 2025. Sea: Low-resource safety alignment for multimodal large language models via synthetic embeddings. *arXiv preprint arXiv:2502.12562* (2025).

[20] Xinbei Ma, Yiting Wang, Yao Yao, Tongxin Yuan, Aston Zhang, Zhuosheng Zhang, and Hai Zhao. 2024. Caution for the environment: Multimodal agents are susceptible to environmental distractions. *arXiv preprint arXiv:2408.02544* (2024).

[21] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. 2025. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326* (2025).

[22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.

[23] Rita Ramos, Bruno Martins, Desmond Elliott, and Yova Kementchedjhieva. 2023. Smallcap: lightweight image captioning prompted with retrieval augmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2840–2849.

[24] Amirhossein Razavi, Mina Soltangheis, Negar Arabzadeh, Sara Salamat, Morteza Zihayat, and Ebrahim Bagheri. 2025. Benchmarking prompt sensitivity in large language models. In *European Conference on Information Retrieval*. Springer, 303–313.

[25] Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, et al. 2025. A survey on (m) llm-based gui agents. *arXiv preprint arXiv:2504.13865* (2025).

[26] Le Wang, Zonghao Ying, Tianyuan Zhang, Siyuan Liang, Shengshan Hu, Mingchuan Zhang, Aishan Liu, and Xianglong Liu. 2025. Manipulating Multimodal Agents via Cross-Modal Prompt Injection. *arXiv preprint arXiv:2504.14348* (2025).
[27] Weihan Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Song XiXuan, et al. 2024. Cogvlm: Visual expert for pretrained language models. *Advances in Neural Information Processing Systems* 37 (2024), 121475–121499.
[28] Xilong Wang, John Bloch, Zedian Shao, Yuepeng Hu, Shuyan Zhou, and Neil Zhenqiang Gong. 2025. EnvInjection: Environmental Prompt Injection Attack to Multi-modal Web Agents. *arXiv preprint arXiv:2505.11717* (2025).
[29] Xuehui Wang, Zhenyu Wu, JingJing Xie, Zichen Ding, Bowen Yang, Zehao Li, Zhaoyang Liu, Qingyun Li, Xuan Dong, Zhe Chen, et al. 2025. MMBench-GUI: Hierarchical Multi-Platform Evaluation Framework for GUI Agents. *arXiv preprint arXiv:2507.19478* (2025).
[30] Chen Henry Wu, Rishi Rajesh Shah, Jing Yu Koh, Russ Salakhutdinov, Daniel Fried, and Aditi Raghunathan. [n. d.]. Dissecting Adversarial Robustness of Multimodal LM Agents. In *The Thirteenth International Conference on Learning Representations*.
[31] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. 2024. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218* (2024).
[32] Cihang Xie, Jianyu Wang, Zhishuai Zhang, Zhou Ren, and Alan Yuille. 2017. Mitigating adversarial effects through randomization. *arXiv preprint arXiv:1711.01991* (2017).
[33] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. 2025. Mobile-agent-v3: Foundamental agents for gui automation. *arXiv preprint arXiv:2508.15144* (2025).
[34] Mang Ye, Xuankun Rong, Wenke Huang, Bo Du, Nenghai Yu, and Dacheng Tao. 2025. A survey of safety on large vision-language models: Attacks, defenses and evaluations. *arXiv preprint arXiv:2502.14881* (2025).
[35] Kaiyuan Zhang, Mark Tenenholtz, Kyle Polley, Jerry Ma, Denis Yarats, and Ninghui Li. 2025. BrowseSafe: Understanding and Preventing Prompt Injection Within AI Browser Agents. *arXiv preprint arXiv:2511.20597* (2025).
[36] Xiaoyu Zhang, Cen Zhang, Tianlin Li, Yihao Huang, Xiaojun Jia, Ming Hu, Jie Zhang, Yang Liu, Shiqing Ma, and Chao Shen. 2023. Jailguard: A universal detection framework for llm prompt-based attacks. *arXiv preprint arXiv:2312.10766* (2023).
[37] Yanzhe Zhang, Tao Yu, and Diyi Yang. 2024. Attacking vision-language computer agents via pop-ups. *arXiv preprint arXiv:2411.02391* (2024).
[38] Haoren Zhao, Tianyi Chen, and Zhen Wang. 2025. On the robustness of gui grounding models against image attacks. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 1618–1623.
[39] Yunqing Zhao, Tianyu Pang, Chao Du, Xiao Yang, Chongxuan Li, Ngai-Man Man Cheung, and Min Lin. 2023. On evaluating adversarial robustness of large vision-language models. *Advances in Neural Information Processing Systems* 36 (2023), 54111–54138.
[40] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *International conference on machine learning*. PMLR, 12697–12706.
[41] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614* (2024).